


STREAMING

DOC NO. RL-370-STREAMING-A REV. A RIPOSTE LABORATORIES

Short answer: yes, at segment granularity, and the round trip stays byte-exact. Tested end to end. This document is what was measured, what changed to support it, and what is still missing.

CONTENTS

▸ Why it works	1
▸ The shape that works: split on GOP boundaries	2
▸ The thing that will bite you: keystream reuse	3
▸ You do not need to transmit a manifest	4
▸ Public-key sessions	5
▸ Latency	5
▸ Throughput	6
▸ What has been verified	6
▸ What is not done	7

SEC.01

WHY IT WORKS

The transform was already frame-local. A frame's plan is derived from `HMAC-SHA256(key, nonce || feature || segment || stream || frame)` and nothing else — no chaining, no running state, no dependency on any other frame. Two frames can be locked on two different machines in either order and the result is the same.

So the cipher was never the obstacle. The obstacle is I/O: FFedit has to see a complete, decodable file before it can export anything, and a live stream never ends.

SEC.02

THE SHAPE THAT WORKS: SPLIT ON GOP BOUNDARIES

Encode the carrier with closed GOPs and every GOP begins with its own sequence header, which makes each GOP a self-contained decodable unit:

CONSOLE

```
$ glitchlock prepare live.mkv -o carrier.mpg --gop 12 --closed-gop
```

Then split the byte stream on `00 00 01 B3`, lock each piece with its own segment number, and concatenate. `glitchlock.stream` provides the splitter:

PYTHON

```
from glitchlock.stream import SegmentReader
from glitchlock.core import lock

reader = SegmentReader()
```

```
n = 0
for chunk in incoming_bytes():      # any size, boundaries don't matter
    for segment in reader.feed(chunk): # complete GOPs only
        n += 1
        lock(segment_path, out_path, key=KEY, nonce=NONCE,
              features=["mv", "qscale"], segment=n, selftest=False)
```

The receiver needs no side channel. It finds the same boundaries by scanning the locked stream for the same sequence headers – verified in the test suite, where the receiver's split is asserted to match the sender's byte for byte.

SEC.03 THE THING THAT WILL BITE YOU: KEYSTREAM REUSE

Each segment is a fresh document, so its frame numbering restarts at zero. Lock segments independently without saying so and **every segment's frame 0 derives an identical plan from the same key and nonce**. Measured on two different GOPs of a real carrier:

```
chunk 0  label mv|0|1|plan  slots=600
         first offsets      : [60, 59, 9, 57, 15, 4, 19, 63, 42, 21]
         first permutation  : [504, 390, 9, 100, 515, 307, 149, 389, 91, 386]
chunk 5  label mv|0|1|plan  slots=600
         first offsets      : [60, 59, 9, 57, 15, 4, 19, 63, 42, 21]
         first permutation  : [504, 390, 9, 100, 515, 307, 149, 389, 91, 386]
```

Identical. That is textbook keystream reuse across the whole broadcast, and it is exactly the kind of failure that looks fine – the round trip still works – while quietly destroying the security argument.

`lock(..., segment=n)` fixes it by adding a fifth field to the domain-separation label. Segment 0 keeps the original four-field label so ordinary single-file locks are unchanged; the two forms differ in field count and can never collide. **Give every segment of a stream a distinct, monotonically increasing number.**

SEC.04 YOU DO NOT NEED TO TRANSMIT A MANIFEST

For a stream, the manifest degenerates to a session parameter set. The receiver knows the key, the nonce, the feature list and the mode from the session, and it can count its own segment number from its position in the stream – so it can synthesise the manifest locally. Verified: segments locked normally, then recovered byte-exact against a Manifest the receiver constructed from nothing but shared parameters and a counter.

That removes the per-chunk sidecar entirely. What you give up:

- ▶ **The digest checks.** `carrier_sha256` / `locked_sha256` and the manifest MAC are per-file values; a synthesised manifest has none, so a corrupted or substituted segment is not detected. Add your own integrity layer if you need one.
- ▶ **Repairs.** A synthesised manifest carries an empty repair map. That is always correct for `mv` and `qscale` – the repair list has come back empty in every measurement – but if a segment ever did need one, the receiver would silently mis-restore those slots. A sender should check `layer.repairs` and refuse to run in manifest-free mode if it is ever non-empty.

SEC.05

PUBLIC-KEY SESSIONS

Recipients compose with streaming, but do the asymmetric work **once per session**, not once per segment:

1. Seal a random session key to the subscriber's public key. Measured at 0.22 ms, producing a 241-byte handshake blob.
2. Send that blob once, out of band or as a stream header.
3. Lock every segment under the session key with its own segment number.
4. The subscriber unseals once (0.07 ms) and then unlocks every segment from the session key plus its own counter.

Verified end to end: all segments recovered byte-exact, and an identity that is not a recipient is rejected at the handshake.

Sealing per segment would also work and costs little, but it puts a KEM block in front of every GOP and gains nothing – the segment numbers already separate the keystreams. One handshake is the right shape, and it is what real streaming crypto does.

Note that reusing one session key across segments is safe **only** because the segment numbers differ. That is the same trap as above, wearing a different hat.

SEC.06

LATENCY

One segment, by construction: a segment is only complete once the next one begins. At GOP 12 and 25 fps that is 480 ms of buffering, plus processing. Shorter GOPs trade compression efficiency for latency.

SEC.07

THROUGHPUT

MPEG-2, GOP 12, `mv + qscale`, one segment per process, on 4 cores of an i9-12900H. "Real time" is segment duration over wall clock – above 1.0 means it keeps up with a live feed.

RESOLUTION	LOCK, 1 WORKER	LOCK, 4 WORKERS	UNLOCK, 1 WORKER	SIZE GROWTH
640×480	2.1×	—	2.8×	1.15×
1280×720	0.74×	1.58×	0.98×	1.24×
1920×1080	0.33×	0.70×	1.1× (est.)	1.30×

Segments are independent, so this parallelises across processes with no coordination – scaling stopped at 4 workers here only because the container has 4 cores. Read it as: **SD is comfortable, 720p works with 4 cores, 1080p needs more** than were available for this test.

Locked streams are 15–30% larger than the carrier, because scrambled vectors are less predictable and cost more bits. Budget for it on the transport.

SEC.08

WHAT HAS BEEN VERIFIED

- ▶ Every GOP segment of a closed-GOP carrier decodes standalone.
- ▶ Lock each segment → concatenate → the joined stream decodes end to end.
- ▶ The receiver re-derives segment boundaries from the locked stream alone.
- ▶ Concatenated restore is byte-identical to the original carrier.
- ▶ The same segment content under two segment numbers produces different output.
- ▶ FFedit runs on `pipe:0` → `pipe:1` with a byte-exact identity apply, so the pieces can be moved through a pipeline without touching disk.

SEC.09

WHAT IS NOT DONE

- ▶ **No transport wrapping.** This works on an MPEG-2 **e**lementary stream. Real delivery is MPEG-TS, HLS or RTMP, which wrap the ES in a container. TS needs a de-mux → lock ES → remux step. HLS is the friendlier target, since segments are already GOP-aligned – lock each segment's video ES and rewrite the segment.
- ▶ **No audio.** prepare drops it. A real chain has to carry audio around the locked video path and re-mux.
- ▶ **No daemon.** There is no long-running process, no socket handling, no back-pressure, no recovery from a mid-segment disconnect.
- ▶ **1080p real-time** needs more cores than were tested, or a faster path.
- ▶ **A named-pipe input silently produced an empty export** in one probe while still exiting zero. Anything built on FIFOs should check output size rather than trusting the exit code.

SOURCE STREAMING.md SHA 1ecb1a88c6fd BUILT 2026-08-14